

Lecture 7: Asynchronous programming

Presentations students, introduction to async

Willem Vanhulle

DevLab Rust 2025

Tuesday December 9, 2025

github.com/wvhulle/rust-course-ghent

Students will present their work.

2.1. Thread parking

Let's assume you have access to threads.

Info

For this presentation we will assume a standard desktop operating system.

Threads can be “parked”.

2.2. Unparking threads

However, the thread itself needs to call `thread::park()` and then the spawning thread needs to save the join handle and call `unpark` on the join handle later on.

2.3. Thread management

This requires a lot of bookkeeping by the developer.

- needs to think about how many threads do I need
- which threads are parked and need to be unparked
- i need to take data from one thread from the join handle and move it into a new thread

Very often we don't actually need full-blown threads.

Threads are good for problems that need to be solve in parallel.

2.4. Parallelism or something else?

Real parallelism means two different processes can run completely independently.

Think about what kind of problems are truly parallel in real-life?

That is quite rare.

Usually they should start and end at the same time.

2.5. Synchronisation

Many parallel task cannot be completed without synchronisation of shared state:

- Channels
- Mutexes
- Barriers

Very often the complexity of the code increases in parallel programming because of synchronisation and performance does not get any better.

Definition

Concurrency is the ability to advance multiple tasks simultaneously.

Why do managers do micromanagement?

Definition

Concurrency is the ability to advance multiple tasks simultaneously.

Why do managers do micromanagement?

They want to reduce risk and lack of control. They want to save money on developers and have more money in their pocket. Developers may **get sick or leave together with their knowledge**.

They do not trust their employees.

Similarly a developer might want to spawn several processes or thread.

Sometimes it is better not to do micromanagement.

What is a way to solve the issues caused by micro-management?

2.6. Concurrency with a metaphor

Definition

Concurrency is the ability to advance multiple tasks simultaneously.

Why do managers do micromanagement?

They want to reduce risk and lack of control. They want to save money on developers and have more money in their pocket. Developers may **get sick or leave together with their knowledge**.

They do not trust their employees.

Similarly a developer might want to spawn several processes or thread.

Sometimes it is better not to do micromanagement.

What is a way to solve the issues caused by micro-management?

Give more responsibility to your developers.

Similarly, let your threads in your large application talk to each other.

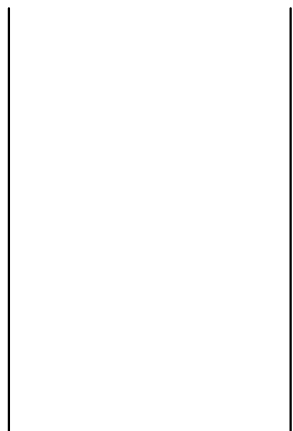
In asynchronous programming and in developer teams means saying “i am stuck”.



2.7. Back to code

Similarly, let your threads in your large application talk to eachother.

In asynchronous programming and in developer teams means saying “i am stuck”.



So asynchronous programming is just multiple threads of computation working cooperatively and telling eachother when they get stuck.

By telling eachother when they get stuck, other threads can jump in and spend precious compute time.

Similarly, a good boss would not halt the whole company because of one team under-performing.

2.8. Coroutines

Coroutines are functions that may yield an intermediate result.

Asynchronous functions in Rust are a special case.

The runtime model of async functions is equivalent to and implement by state machines.

2.9. What Rust does differently

The state machines are by default allocated on the stack. When the state machine goes to the next state, the async runtime does not move the state machine.

In terms of code, this means that local variables in async functions before the await are not moved anyway.

In other languages the Runtime has to copy the whole set of local variables to another location in RAM memory to be able to run another async function. This is less efficient.

3.1. Rust tooling

Pinning Rust version in `rust-toolchain.toml`:

```
1 [toolchain]
2 channel = "1.91"
3 components = ["rustfmt", "clippy", "rust-analyzer", "rust-src"]      (playground link)
```

In NixOS:

```
1 fenix = {
2     url = "github:nix-community/fenix";
3     inputs.nixpkgs.follows = "nixpkgs";
4 };                                (playground link)
```

```
1 rustToolchain = fenix.packages.x86_64-linux.fromToolchainFile {
2     file = ./rust-toolchain.toml;
3     sha256 = "sha256-SDu4snEWjuZU475PERvu+i050Mi39KVjqCeJeNvpguU=";
4 };                                (playground link)
```

```
1  pkgs.mkShell {
2      nativeBuildInputs = [
3          pkgs.pkg-config
4          rustToolchain
5      ];
6
7      buildInputs = [
8          pkgs.openssl
9      ];
10
11      LD_LIBRARY_PATH = pkgs.lib.makeLibraryPath [ pkgs.openssl ];
12
13      shellHook = ''
14          echo "Welcome to the Rust course!"
15          '';
16  };
```

(playground link)